

A/B 预测算法设计策划案

当前版本：v1.0

● 文档历史

文档版本	日期	修订者	说明
v1.0	2026.7.28	Kaka	初始文档

● 文档约定

文字颜色	表示
黑色	已确定的部分
灰化	暂时挂起、搁置
蓝色	本次版本更新部分
红色	重点内容
楷体	解释说明的内容

目录

A/B 预测算法设计策划案	1
目录	2
方案一：AMM 模式（常数乘积算法）- polymarket 早期核心算法	3
一、 核心数学模型参数定义	3
二、 平台初始开盘（初始化公式）	4
步骤 1：生成等额的初始份额库存	4
步骤 2：锁定流动性常数 K	4
步骤 3：确立初始瞬时报价	5
三、 运行期核心计算公式（用户买入）	5
步骤 1：扣除手续费，计算净投入	5
步骤 2：计算用户能获得的 YES 份额数量	5
步骤 3：更新资金池库存（YES 变少，价格自动变贵）	5
步骤 4：计算下一位用户的瞬时报价	6
方案二：帕利 mutuel 奖池模式（Pari-mutuel）	7
一、 核心参数定义	7
二、 运行期核心计算公式	8
步骤 1：扣除手续费，计算净投注	8
步骤 2：实时更新单边与总奖池	8
步骤 3：实时显示当前赔率与名义价格	8
三、 终局结算与分钱公式（开奖时）	9
情况 1：最终结果为 YES 胜出	9
情况 2：最终结果为 NO 胜出	9

方案一：AMM 模式（常数乘积算法） - polymarket 早期核心算法

核心玩法：实时买卖预测代币（类似去中心化交易所炒币）

爽点（好处）：

- **刺激好玩：**像炒股一样能追涨杀跌、随时买卖，平台狂刷手续费。
- **价格锁死：**买时赔率是多少就固定是多少，支持提前割肉或止盈离场。
- **拓展性强：**自带代币属性，以后方便加杠杆或做金融衍生玩法。

坑点（坏处）：

- **平台要垫资：**开盘得放“启动资金”。行情一边倒时，平台可能亏掉本金。
- **开发难度大：**数学模型复杂、周期长，还要防最后 1 秒大资金偷鸡刷单。

一、 核心数学模型参数定义

M_seed：平台官方真实拨出并锁死在池子里的“种子基金”总量（平台币，例如 1000 币）。

S_yes：当前资金池内剩余可购买的 YES 份额库存。

S_no：当前资金池内剩余可购买的 NO 份额库存。

K：流动性常数（开盘初始化时锁死，全生命周期保持 $S_yes \times S_no = K$ ）。

M：外来用户实际支付的平台币总量。

f：平台手续费率（例如 2%，即 $f = 0.02$ ）。

M_{net}: 扣除手续费后, 真正进入资金池的外来平台币。

P_{yes}: 下一位用户买入 YES 的瞬时报价 (单位: 平台币/份额)。

二、 平台初始开盘 (初始化公式)

在事件刚发布、没有任何外来用户买入时, 系统必须使用平台拨出的种子基金

M_{seed} 进行初始化:

步骤 1: 生成等额的初始份额库存

平台投入 M_{seed} 个真实平台币, 由于开盘时 YES 和 NO 的概率对等 (各占 50%, 初始价格均为 0.5 币), 系统将在池子内自动 1 比 1 生成等额的份额数量:

- **初始 YES 份额库存** (S_{yes_init}) = M_{seed}
- **初始 NO 份额库存** (S_{no_init}) = M_{seed}

(注: 例如平台出 1000 币种子基金, 池子里就分别放入 1000 个 YES 和 1000 个 NO 的份额。)

步骤 2: 锁定流动性常数 K

计算并锁死常数 K 的值:

$$K = S_{yes_init} \times S_{no_init}$$

(注: 在后续所有交易中, 无论两边份额如何变动, **K 值永远保持不变**。)

步骤 3：确立初始瞬时报价

根据池子库存比例，初始买入 1 个 YES 份额的瞬时价格完美锚定在 0.5 币：

$$P_{yes_init} = S_{no_init} / (S_{yes_init} + S_{no_init}) = M_{seed} / (M_{seed} + M_{seed}) = 0.5$$

三、 运行期核心计算公式（用户买入）

初始化完成后，盘口开始接待外来用户。

步骤 1：扣除手续费，计算净投入

外来用户扔 M 币进来买 YES 时，计算净投入：

$$M_{net} = M \times (1 - f)$$

步骤 2：计算用户能获得的 YES 份额数量

用户塞入 M_{net} 平台币，资金池为了维持常数 K 的平衡，会吐出对应的 YES 份额给用户：

$$\Delta S_{yes} = S_{yes} - [K / (S_{no} + M_{net})]$$

步骤 3：更新资金池库存

交易完成后，池子里的份额数量更新为：

- $S_{no_new} = S_{no} + M_{net}$
- $S_{yes_new} = K / S_{no_new}$

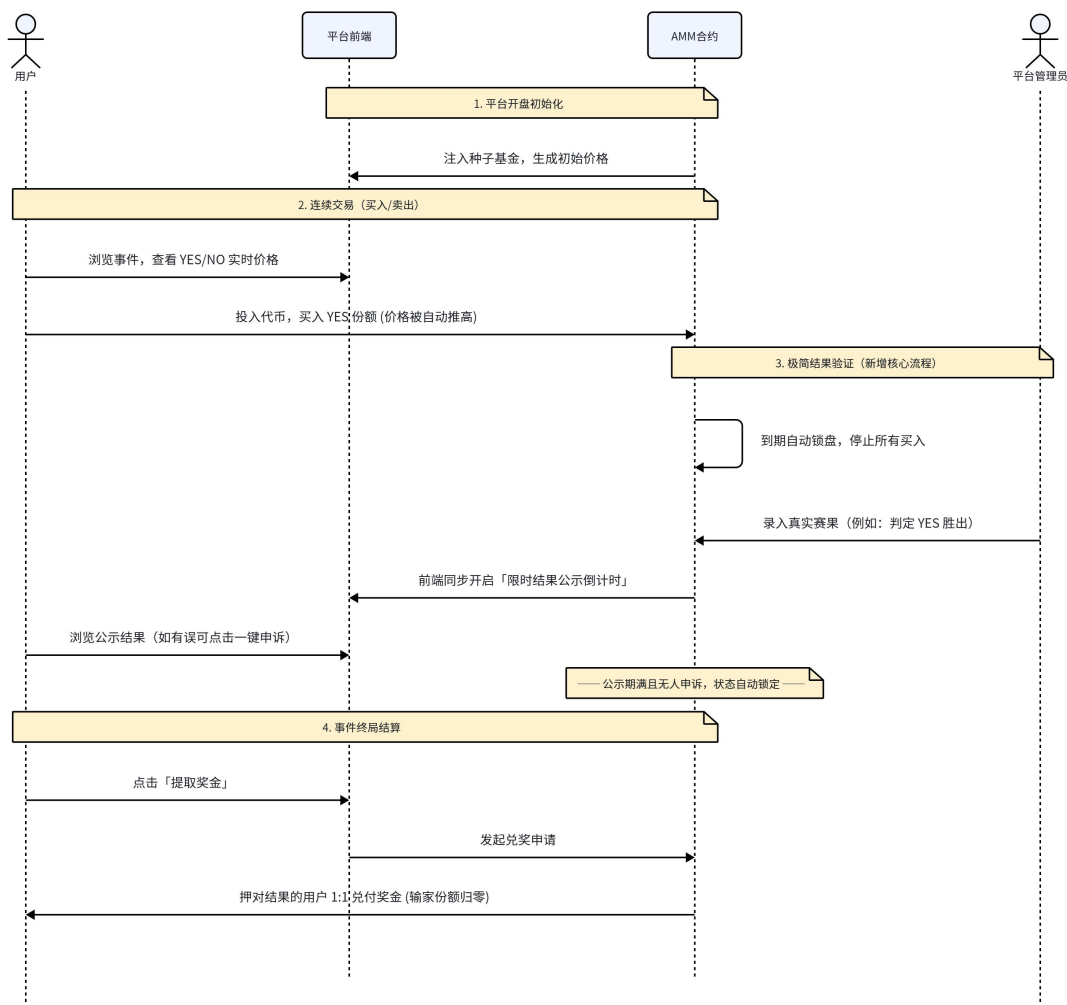
步骤 4：计算下一位用户的瞬时报价

公式更新后，后续新用户买入 YES 的瞬时价格更新为：

$$P_{yes} = S_{no_new} / (S_{yes_new} + S_{no_new})$$

注：用户如果买 no，只需要更换步骤 2，3 的公式 yes 为 no 即可。

时序图（技术流程图）



方案二：帕利 mutuel 奖池模式 (Pari-mutuel)

核心玩法：最纯粹的共享大奖池（买 YES 的和买 NO 的互相对赌，赢了分输家的钱）

爽点（好处）：

稳赚不赔：平台不用垫资，纯靠用户互掐，平台坐收无风险抽水。

开发贼快：只有简单的加减乘除，几天就能上线拿去测试市场。

坑点（坏处）：

赔率不固定：不到开奖那一秒不知道能赢多少，用户反馈差、没安全感。

盘口容易死：钱投进去就被锁死，不能中途退赛，玩久了缺乏活跃度。

一、 核心参数定义

Total_Pool：当前奖池内的平台币（或代币）总投注额。

Pool_YES：当前投注在 YES（会发生）上的平台币总额（初始=0.01）。

Pool_NO：当前投注在 NO（不会发生）上的平台币总额（初始=0.01）。

M：当前用户本次新投入的投注额（平台币）。

f：平台抽水率/手续费率（例如 5%，即 $f = 0.05$ ）。

M_net：扣除手续费后，真正进入奖池的净投注额。

二、 运行期核心计算公式

步骤 1：扣除手续费，计算净投注

用户下注 M 币，**计算净投注：**

- $M_{net} = M * (1 - f)$

步骤 2：实时更新单边与总奖池

根据用户下注的选项，更新对应的池子数据。

- 如果用户买 YES：

$$Pool_YES_新 = Pool_YES_旧 + M_{net}$$

- 如果用户买 NO：

$$Pool_NO_新 = Pool_NO_旧 + M_{net}$$

- **总奖池实时更新（两边净投注相加）：**

$$Total_Pool = Pool_YES + Pool_NO$$

步骤 3：实时显示当前赔率与名义价格

- **YES 的当前赔率** = $Total_Pool / Pool_YES$
- **NO 的当前赔率** = $Total_Pool / Pool_NO$
- **YES 的当前名义价格（概率占比）** = $Pool_YES / Total_Pool$

(两边价格相加永远等于 1)

三、 终局结算与分钱公式（开奖时）

当事件结果揭晓时，系统根据胜出方进行一键清算，输家池子里的钱全部按比例分给赢家。

情况 1：最终结果为 YES 胜出

所有投 NO 的钱归 YES 赢家所有。每个投 YES 的用户，根据自己当时占 YES 总池子的投注比例，瓜分整个总奖池（Total_Pool）：

- 用户分得的平台币 = $\text{Total_Pool} * (\text{该用户的 YES 净投注额} / \text{Pool_YES})$

情况 2：最终结果为 NO 胜出

所有投 YES 的钱归 NO 赢家所有。每个投 NO 的用户，根据自己当时占 NO 总池子的投注比例，瓜分整个总奖池（Total_Pool）：

- 用户分得的平台币 = $\text{Total_Pool} * (\text{该用户的 NO 净投注额} / \text{Pool_NO})$

时序图（技术流程图）

